Indian Institute of Technology, Kharagpur
Department of Computer Science and Engineering

Programming and Data Structure (CS10003)
Class Test 1, Autumn 2026

Full marks: 30 Date: 02-Sep-2026

Answer the questions in the spaces provided (within the boxes or blanks). You may use other empty spaces for rough work, but your final answer must be contained within the given boxes / blank spaces. No additional sheet will be given.

Roll Number:		**Section** (1/2/3/4):	
Name:			

Question:	1	2	3	4	Total
Full marks:	7	8	7	8	30
Marks Obtained:					

1. (7 marks) Answer the following questions.

   (a) (2 marks) Which of the following are VALID variable names in C? **Tick only the valid ones**: (i) AVG_ (ii) _AVG (iii) AVG2 (iv) 2AVG (v) AVG_NUM (vi) AVG-NUM
   Valid: (i) AVG_ (ii) _AVG (iii) AVG2 (v) AVG_ NUM (0.5 for each)

   (b) (2 marks) Assume that a variable of type *short* occupies 2 bytes on a particular machine. On this same machine:
   (i) How many bytes will an *unsigned short* variable s occupy? **2 bytes**
   (ii) What is the maximum value that s can store? $2^{16} - 1$ **or 65,535**

   (c) (3 marks) What is the output of the following program? Write the *exact* output in the box below.

```
main(){
    int a=10, b=30, x, y;   float f;
    x = a++  +  --b;
    y = b--  -  ++a;
    f = x / y;
    printf("x = %d, y = %d, f = %f\n", x, y, f);
}
```

x = 39, y = 17, f = 2.000000 (1 mark for each value)

2. (8 marks) Answer the following questions.

(a) ($4 \times 0.5 + 1 = 3$ marks) What would the following program print?

```
#include <stdio.h>
int main() {
    int a = 7, b = 3, c;
    float x = 1.6, y;

    c = (a > b) + (a == b) + (a < b);
    printf("c = %d\n", c);

    y = (a / b) + (a % b) * x - (b / a);
    printf("y = %f\n", y);

    a = b = c = (a > 5) ? a - b : b - a;
    printf("a=%d b=%d c=%d\n", a, b, c);

    int p = 4, q = 0, r = 9;
    if ((q = p + r, q > 10) && (r = r - p))
        printf("branch1: q=%d r=%d\n", q, r);
    else
        printf("branch2: q=%d r=%d\n", q, r);

    int s = 5, t = 8, u;
    u = (s > t) ? 1 : (s < t) ? -1 : 0;
    printf("s=%d t=%d u=%d\n", s, t, u);
    return 0;
}
```

Output:

c=1
y=3.600000
a=4 b=4 c=4
branch1: q=13 r=5
s=5 t=8 u=-1

(0.5 marks each for lines 1,2,3,4 and 1 mark for the last line since it tests nested ternary correctly)

(b) ($1 + 2 = 3$ marks) A 4-digit number `a` ($1000 \leq a \leq 9999$) with digits $d_1d_2d_3d_4$ (d_1 is the thousands digit, d_4 is the units digit) is called **Regal** iff *all three* of the following hold:

(i) it is *Harshad*: divisible by its digit sum $d_1 + d_2 + d_3 + d_4$;

(ii) its digit sum is *prime*;

(iii) it is a *palindrome*: The numbers read the same from left to right and right to left.

In the program given below, write (1) an expression `s` that computes the sum of digits of `a`, and (2) another expression `isRegal` that is true if and only if `a` is Regal (you can use `s` to write this expression). Both expressions should be written (in the provided box) *without using conditional statements like if-else, loops, or functions.*

[Hint: The set of prime numbers within a certain range is limited.]

```c
#include <stdio.h>
int main() {
    int a = -1;
    while (a < 1000 || a > 9999) {
        scanf("%d", &a);
    }

    // sum of digits of a
    int s = ....;

    // isRegal  (see definition above)
    int isRegal = ....;

    if (isRegal) printf("%d is Regal\n", a);
    else printf("%d is NOT Regal\n", a);
    return 0;
}
```

s =

isRegal =

s = a/1000 + (a/100)%10 + (a/10)%10 + a%10

isRegal = (a % s == 0) &&
(s==2||$s == 3||s == 5||$
$s == 7||s == 11||s == 13||$
$s == 17||s == 19||s == 23||$
$s == 29||s == 31$)
&&$(a/1000 == a\%10)$
&&$((a/100)\%10 == (a/10)\%10)$;

ALTERNATIVELY

$isRegal = 0$

(1 mark for s
2 marks for isRegal: 0.5 Harshad term, 0.5 prime digit-sum term, 0.5 for palindrome term, 0.5 for correctly combining all three with &&)
ALTERNATIVELY
2 marks for 0 since isRegal is always false

(c) ($4 \times 0.5 = 2$ marks) Trace the code below for three separate runs: `x=2, y=3`; `x=1, y=1`; `x=5, y=9`. Give the exact output printed in each case. Then answer: is the `default` label ever reached *directly* (i.e. by the switch matching `v` to no other case), or only reached by falling through from `case 3`? Justify briefly (no marks for just writing yes/no without correct explanation).

```
int x, y, v;
// (x, y) is as shown above for each run
v = (x + y) % 5;
switch (v) {
    case 0: printf("A");
    case 1: printf("B"); break;
    case 2:
    case 3: printf("C");
    default: printf("D"); break;
    case 4: printf("E");
}
```

```
x=2,y=3: Output:
x=1,y=1: Output:
x=5,y=9: Output:
Is default reached directly
(Yes/No)?
Why (one sentence):
```

x=2,y=3 (v=0): AB
x=1,y=1 (v=2): CD
x=5,y=9 (v=4): E
default can be reached directly when x+y is negative and not a multiple of 5.
(0.5 each for 3 outputs, 0.5 for default + explanation)

3. (7 marks) Answer the following questions:

   (a) (3 marks) The program below is meant to read n, and print the total area of circles of radius $1, 2, \cdots, n$. It contains three errors. Identify each one and give the correction.

```
#include <stdio.h>
#define AREA(r)  3.14*r*r

int main(){
    int i, n;
    float total = 0.0;
    scanf("%d", &n)

    for (i = 0; i < n; i++)
        total = total + AREA(i+1);
    printf("Total area = %d\n", total);

    return 0;
}
```

```
1.
2.
3.
```

1. **Missing semicolon after `scanf("%d", &n)`.**
2. **`%d` used to print `total`, which is a `float` – use `%f` instead.**
3. **`AREA(i+1)` expands to `3.14*i+1*i+1`, which is not the area, since the macro is plain text substitution. Write `#define AREA(r) (3.14*(r)*(r))` instead.**

   (b) (4 marks) Write the exact output of the following program in the provided box:

```c
#include <stdio.h>
int x = 10;

int f(int x) {
    static int cnt = 0;
    cnt++;
    x = x + cnt;
    printf("f: x = %d,count = %d\n", x, cnt);
    return x;
}

void g(void) {
    x = x + 5;
    printf("g: x = %d\n", x);
}

int main() {
    int y = 3;
    y = f(y);
    g();
    y = f(x);
    printf("main: x = %d,y = %d\n", x, y);
    return 0;
}
```

```
f:  x = 4, count = 1
g:  x = 15
f:  x = 17, count = 2
main:  x = 15, y = 17
```

4. (8 marks) Answer the following questions:

   (a) (2 marks) What is the error (if any) in the program? If there is no error, write "No error"; if there is an error, describe the error in the provided box.

```c
#include<stdio.h>

int main(){
    int i=1;
    while(i<=10){
        printf("%d\n", i);
        i--;
    }
    return 0;
}
```

The loop never ends. i is decremented instead of i++, so $i <= 10$ stays true and it is an infinite loop.

(b) (3 marks) Write the exact output produced by the following program.

```
#include<stdio.h>

int main(){
    int i, j;
    for(i=1; i<=3; i++){
        for(j=1; j<=3; j++){
            if(j==i)
                continue;
            printf("%d ", i*j);
        }
        printf("\n");
    }
    return 0;
}
```

Output:

2 3
2 6
3 6

(1 mark per line)

(c) (3 marks) What does the following program compute? Give the output for `n=1234` and show the values of `n` and `out` over the iterations.

```
#include<stdio.h>
int main(){
    int n=1234, out=0;
    while(n > 0){
        out = out*10 + n%10;
        n /= 10;
    }
    printf("OUT = %d", out);
    return 0;
}
```

Computes: **reverse of n**

Output: **REV = 4321**
(1 mark)

n & out after each iter:

out=4, n=123
out=43, n=12
out=432, n=1
out=4321, n=0
(0.5*4 = 2 marks)

[Extra space for rough work]

[Extra space for rough work]